

IDP Pipeline — Technical Report

Bluno Intern Assignment — IDP Service
LangGraph + FastAPI + Groq LLM
Raj Prajwal M | work.rajprajwal@gmail.com

1. Overview

This report documents the design, implementation, and evaluation of an Intelligent Document Processing (IDP) service built as part of the Bluno intern assignment. The service accepts PDF uploads via a REST API and returns structured JSON containing all extracted fields, per-field confidence scores, a human-in-the-loop review flag, line items (for invoices), and per-node processing times.

Three document types are supported: **commercial invoices**, **sea waybills**, and **airway bills**. The pipeline is built on **LangGraph StateGraph** with typed state, conditional routing, and parallel PDF parsing. A **/batch** endpoint allows up to 20 concurrent uploads via `asyncio.gather`.

2. Pipeline Architecture

All documents pass through a four-stage LangGraph pipeline. Each node reads from and writes back to a shared **IDPState** TypedDict — nodes communicate only through this structure, with no global variables or side channels.

Node	Role	Key Design Choices
parse	Dual parsing	docling (layout-aware, local) + LlamaParse (cloud OCR) run concurrently. Outputs are MERGED (concatenated) — not primary+fallback. Both parsers fail → <code>is_fatal_error=True</code> , graph exits early.
classify	Document type routing	Keyword scoring first (fast, free). LLM only if no keyword hits. Routes to <code>extract_invoice</code> , <code>extract_transport</code> , or <code>error_node</code> .
extract	Field extraction	Hybrid: rule-based (regex) first, then one batched LLM call for all low-confidence fields. Invoice: also extracts line items. Transport: form-layout fields sent directly to LLM with explicit prompt to ignore template labels.
validate	Pydantic validation + HITL	Coerces each field individually — one field failing ≠ full rejection. Attaches per-field confidence scores, per-node timings, and HITL flag.

ASCII Pipeline Diagram

```
PDF Upload ▀ ▾ [parse] ← docling (layout) + LlamaParse (OCR) — merged ▀ ▾
[classify] ← keyword scoring → LLM fallback ▀ ▀▀▀ invoice ▀▀▀ [extract_invoice] →
line items + hybrid LLM ▀▀▀ seaway/airway ▀ [extract_transport] → LLM for form
fields ▀▀▀ unknown/error ▀ [error_node] ▀ ▾ [validate] ← Pydantic + HITL flagging ▀
```

3. Key Design Decisions

Dual parser merge (not fallback)

Both docling and LlamaParse run concurrently via `asyncio.gather`. Their text outputs are concatenated with a separator so neither parser's layout gaps affect extraction — docling's structure plus LlamaParse's OCR coverage are combined in every extraction call.

Hybrid extraction — rule-based first, batched LLM fallback

Regex rules run first (no API cost, deterministic). All fields with confidence below 0.7 are collected and sent to Groq in a single batched call. This minimises latency and API cost while ensuring high recall.

LLM-first for transport form layouts

Sea waybills and airway bills use a form layout where label text and value text appear in adjacent cells. Regex inevitably captures template placeholders (e.g. `' / Exporter (complete name and address)'`) instead of actual values. All text fields are sent directly to the LLM with an explicit prompt to ignore template labels; only unambiguous formats (BL numbers, container refs, dates, weights) use rule-based extraction.

Decimal for monetary values

All monetary fields use Python's `Decimal` type (not `float`) to avoid floating-point rounding errors. The `_safe_decimal()` helper converts raw strings individually so one bad monetary field never rejects the whole response.

Partial success principle

Field-level failures produce an `ExtractionError` appended to the errors list, not an HTTP 422. A response with errors is still returned — downstream systems get all successfully extracted fields and can decide what to do with the failed ones.

Errors field never null

The errors list is always present (empty list if no errors). This lets consumers use `errors[0]` without null-checking.

Pydantic v2 date serialisation

`datetime.date` objects are serialised to ISO 8601 strings via `model_dump(mode='json')` at the API layer. Using `'import datetime'` (not `'from datetime import date'`) avoids Pydantic v2 annotation resolution errors.

`load_dotenv()` called first

`dotenv` is loaded before any import in `main.py` to guarantee `GROQ_API_KEY` is in the environment when `LangChain` and `Groq` initialise their clients.

4. Confidence Scoring

Every extracted field carries a confidence score between 0.0 and 1.0. Scores drive two downstream behaviours: (1) whether to call the LLM for a field, and (2) whether to flag the document for human review.

Score	Meaning	Trigger
1.0	Clean regex match	Rule-based pattern matched without ambiguity
0.85	LLM extraction	LLM fallback succeeded
0.7	Heuristic / pattern with noise	Pattern found but required normalisation
0.0	Not found / pending LLM	No rule matched — field goes to LLM batch

The LLM call threshold is **0.7**. Any field with confidence < 0.7 is included in the batched LLM request. After the LLM responds, matched fields are updated to 0.85.

5. Field-Level Precision and Recall

Evaluation was run against **14 invoice PDFs** using `tests/evaluate.py`. Ground truth was manually verified for each file (Aaron Bergman confirmed against provided ground truth; all 14 share the same SuperStore template with invoice numbers embedded in filenames). The server was running locally; each PDF was submitted to the `/extract` endpoint and the response was compared field-by-field against ground truth.

Field	Precision	Recall	TP	FP	FN	Notes
issuer	100%	100%	14	0	0	Rule-based: text before 'Bill To:'
date	100%	100%	14	0	0	Rule-based: label-anchored regex
total	100%	100%	14	0	0	Rule-based: Total line + Decimal
invoice_number	100%	100%	14	0	0	Rule-based: 'Invoice #' pattern
reference_ids	100%	100%	14	0	0	Derived from invoice_number

Overall: 100% precision and recall across all 5 evaluated fields and 14 documents. All fields were extracted by the rule-based layer (confidence 1.0) — the LLM was called only for the recipient field, which has no ground truth label in the test set. No false positives or false negatives were recorded.

Limitation: The evaluation set consists exclusively of invoices from a single template (SuperStore). Precision/recall for transport documents (sea waybills, airway bills) could not be measured quantitatively because no labelled ground truth dataset was available for those document types. Qualitative testing confirmed that key fields (BL number, shipper, consignee, ports, vessel, date) are correctly extracted from the sample sea waybill PDF after the transport extractor rewrite described in Section 8.

6. Line Item Extraction

Invoice responses include a **line_items** array. Each item has four fields: description, quantity, unit_price, and amount. Extraction is two-stage:

- Rule-based: scans for markdown table rows (`| col | col |`) and space-aligned plain-text columns. Confidence = 1.0 if rows found.
- LLM fallback: if the rule layer finds nothing, the invoice text is sent to Groq with a JSON-mode prompt requesting an array of line item objects.

All monetary fields in line items are stored as **Decimal** via the `_safe_decimal()` helper. Individual item parse failures are silently skipped (partial success principle) — a single malformed row never drops the whole line items array.

7. Human-in-the-Loop (HITL) Flagging

Every response includes a **requires_review** boolean and a **review_reasons** list. The `validate` node sets `requires_review=True` when:

- Any key field for that document type has confidence < 0.7.
- One or more fields failed extraction (non-empty errors list).

Document Type	Key Fields Checked
invoice	issuer, total, date, invoice_number
seaway_bill	shipper, consignee, bl_number, port_of_loading
airway_bill	shipper, consignee, awb_number, airport_of_departure

Each review reason is a human-readable string explaining which field triggered the flag and the actual confidence score, e.g.: *'Low confidence on shipper (0%) — value may be incorrect or missing'*. This lets operators triage quickly without reading the full response.

8. Failure Case Analysis

8.1 Transport Document Form-Layout Problem

What failed: The original `extract_transport.py` used label-based regex patterns to extract fields from sea waybills and airway bills. When tested against a real sea waybill PDF, the following garbage values were returned:

Field	Extracted (broken)	Expected
bl_number	No	APLU 090147471
shipper	/ Exporter (complete name and address)	SINOBUILL PETROCHEMICAL...
consignee	/ Consignee (complete name and address)	TO ORDER OF SHIPPER
vessel_name	/ Voyage / Flag	COSCO HAIFA

Root cause: Sea waybills use a two-column form layout where field labels and values appear in adjacent cells. The PDF text extractor linearises these into a stream where label tokens and value tokens are interleaved. Label-based regex matched the template placeholder text ('/ Exporter (complete name and address)') rather than the actual typed value, because both appeared immediately after the same label keyword.

Fix: The transport extractor was completely rewritten. Rule-based extraction is now reserved only for fields with unambiguous formats: BL numbers (carrier prefix + alphanumeric), container references (4-letter + 7-digit pattern like TCLU6016701), weights (numeric + KGS/LBS/MT), measurements (numeric + CBM/M3), and dates (explicit month-name format). All text fields (shipper, consignee, notify party, vessel name, port names, voyage number, description of goods) are sent directly to Groq with an explicit instruction to ignore template labels such as '(complete name and address)' and '/ Voyage / Flag'.

8.2 LLM Returning String 'null' for Optional Fields

What failed: For some invoices, Groq returned the string literal **"null"** (not JSON null) for the `due_date` field when no due date was present. This passed the *value is not None* guard and reached the `validate` node, which tried to parse it as a date and logged: *Could not parse 'null' to a valid date*.

Fix: A sanitisation step was added after the LLM response is parsed. Any string value matching {'null', 'none', 'n/a', 'na', 'undefined', ''} is normalised to Python `None` before being written into `extracted_fields`. This prevents spurious `ExtractionError` entries and removes false HITL flags.

9. Per-Node Processing Times

Every response includes a `processing_time_ms` dict with elapsed time per node, and a `total_time_ms` sum. Times are measured with `time.perf_counter()` inside each node. Representative values from a single-invoice `/extract` call:

Node	Typical Time	Bottleneck?	Notes
parse	800–2000 ms	Yes	LlamaParse cloud round-trip dominates; docling is ~200 ms local
classify	5–15 ms	No	Keyword scoring. LLM only fires if no keywords match (~500 ms)

Node	Typical Time	Bottleneck?	Notes
extract	400–900 ms	Moderate	Groq API call for LLM fallback fields; rule-based only is <10 ms
validate	1–5 ms	No	Pure Python Pydantic coercion

The parse node is the dominant bottleneck due to LlamaParse's cloud OCR latency (typically 600–1800 ms per document). The `/batch` endpoint mitigates this by processing up to 20 documents concurrently via `asyncio.gather`, achieving near-linear throughput scaling up to the rate limit.

10. LLM Choice Justification

The pipeline uses **Groq + llama-3.1-8b-instant** for all LLM inference. This choice was made for three reasons:

- **Speed:** Groq's hardware acceleration delivers tokens at ~750 tokens/second, making the LLM fallback add only 200–400 ms even for multi-field batches.
- **JSON mode:** Groq supports `response_format={'type': 'json_object'}`, guaranteeing structured output without markdown fence stripping or post-processing.
- **Cost:** llama-3.1-8b-instant is free-tier on Groq with generous rate limits, making it practical for evaluation and development without API spend.

The model is configured via the `GROQ_MODEL` environment variable, so swapping to llama-3.1-70b-versatile or a GPT-4o-class model requires only a `.env` change — no code modifications.

11. Known Limitations

- **Single-template evaluation set:** All 14 test invoices share one layout (SuperStore). Performance on invoices from other vendors (different heading positions, multi-page, non-ASCII characters) has not been quantitatively measured.
- **Transport ground truth:** No labelled ground truth exists for sea waybills or airway bills, so precision/recall for those document types cannot be reported.
- **LlamaParse dependency:** The parse node degrades gracefully if LlamaParse is unavailable (falls back to docling only), but fields that rely on OCR quality may have lower confidence.
- **Groq rate limits:** The free tier caps at 30 req/min for llama-3.1-8b-instant. The `/batch` endpoint's `asyncio.gather` fires all LLM calls simultaneously, which can hit this limit with batches larger than ~10 documents.
- **No persistent storage:** Extraction results are returned in the HTTP response only. There is no database or audit trail — re-processing is required if results are lost.